Auditing SHAP Explanations Against Latent Failure Mechanisms in Predictive Maintenance

1st Koshik Debanath
Department of Computer Science
South Dakota State University
Brookings, SD, USA
koshik.debanath@jacks.sdstate.edu

2nd Prakrit Uprety
Department of Computer Science
South Dakota State University
Brookings, SD, USA
prakrit.uprety@jacks.sdstate.edu

3rd Vision Rijal
Department of Computer Science
South Dakota State University
Brookings, SD, USA
vision.rijal@jacks.sdstate.edu

4th Peiyu Li
Department of Computer Science
South Dakota State University
Brookings, SD, USA
peiyu.li@sdstate.edu

***Abstract*—Machine learning models are increasingly used to predict equipment failures in industrial systems, helping reduce downtime and maintenance costs. Explainable AI methods such as SHapley Additive exPlanations (SHAP) identify the features influencing these predictions, but whether those explanations correspond to the physical causes of failure remains unclear. We investigate this problem using the AI4I 2020 dataset, which records individual failure mechanisms alongside overall failure outcomes. We introduce a statistical framework that evaluates how well model explanations align with known failure mechanisms using three complementary attribution measures. To determine whether this alignment exceeds what could occur by chance, we compare the results against 1,000 models trained on randomly shuffled labels. Experiments show that an XGBoost classifier (test F1 = 0.89) produces statistically significant alignment for three failure mechanisms across five random seeds, with supporting results from a random forest and an alternative explanation method. However, explanations for tool wear depend on the SHAP formulation. Controlled experiments also reveal that misleading features can receive substantial attribution despite strong predictive performance, while some evaluation measures become less sensitive as dataset size increases. These findings highlight the need to evaluate explanations beyond predictive accuracy and provide a statistical approach for assessing their consistency with known physical failure mechanisms.**

***Index Terms*—explainable AI, SHAP, predictive maintenance, faithfulness evaluation, model randomization, cyber-physical systems**

I. Introduction

Predictive maintenance (PdM) uses sensor data to predict when a machine is about to fail, so that repairs can be planned before a breakdown. An alarm alone is not enough: the technician also needs to know *why* it was raised, because the reason decides which component is inspected.

To give these reasons, practitioners often explain PdM models with SHAP (SHapley Additive exPlanations) [1]. SHAP splits a single prediction into one contribution per input feature, such as temperature or torque, so that the contributions add up to the model's output. A SHAP plot that highlights sensible sensors is usually taken as evidence that the model has learned the right behavior [2], [3].

This evidence is weak. A model with high accuracy can still depend on a *shortcut*: a feature that is associated with failure in the training data but is not part of the physical process that causes it [4], [5]. Suppose, for example, that one product type is run under heavier load and therefore fails more often. A model may learn to raise alarms for that product type. Its alarms can still be correct, and its SHAP plot will highlight product type, but this tells the technician nothing about which component is wearing out. Reading SHAP plots by eye cannot separate this case from a model that has learned the physics.

This matters most in safety-critical cyber-physical systems, where explanations are used to decide whether a model can be trusted in operation. Accepted without a test, a model that relies on a shortcut can pass review, and its errors appear only when operating conditions change and the shortcut no longer holds. What is missing is a way to *test* an explanation against the known causes of failure, with a clear pass or fail result.

Checking an explanation against the true cause of failure is hard for two reasons:

- **Missing ground truth.** The test needs to know which physical mechanism caused each failure. Industrial datasets rarely record this.
- **A misleading baseline.** An explanation must be compared with what a meaningless model would produce, and the obvious comparison is misleading. Rows that belong to a failure mechanism have unusual values in that mechanism's sensors (for example, overheating rows have a small temperature difference). SHAP values tend to be large at unusual values, whatever the model has learned. We show that a simple test which ignores this effect approves a model trained on randomly shuffled labels, that is, a model that knows nothing about failure (Section V-B).

The AI4I 2020 dataset [6] removes the first obstacle. Its failure label is the logical OR of five separately recorded

TABLE I
HYPOTHESES TESTED IN THIS PAPER AND SHORT ANSWERS.

	Hypothesis	Answer
H1	For each physical mechanism, the model's SHAP credit falls on that mechanism's trigger features, more than chance would allow and more than on other failures.	**Yes** for heat dissipation, power, and overstrain failure, in all seeds and with two explainers and two model families. **Unresolved** for tool wear failure, whose result depends on the explainer.
H2	The audit catches a model that relies on a shortcut sensor.	**Yes**, once the shortcut takes about a fifth to a third of the credit. **No** for a shortcut that takes about a tenth. Accuracy does not reveal the shortcut.
H3	A retrained null is necessary: a baseline that compares random failure rows under the same model cannot tell a trained model from a shuffled-label one.	**Yes.** The same-model baseline approves both models.

failure mechanisms, four of which follow known physical rules. We train a classifier on the combined failure label only and hide the mechanism flags from it. We then use the flags to *audit* the model's SHAP explanations: for each mechanism, we ask whether the model credits the features that physically cause that mechanism. To remove the second obstacle, we compare every audit statistic against 1,000 models retrained on shuffled labels [7], [8]. Fig. 1 summarizes the process.

We frame the study around three hypotheses. Table I states them and gives our short answers, which the rest of the paper supports.

Our main contributions are:

1) **A per-mechanism audit of SHAP explanations.** It combines three simple statistics, out-of-sample SHAP values, a shuffled-label null, and correction for multiple tests (Section III).
2) **Evidence that a simple baseline is misleading.** Comparing random failure rows under the same model approves a model that learned nothing (Section V-B).
3) **Controlled stress tests.** We inject a shortcut sensor of graded strength (Section V-C), and we vary the seed, the rows audited, the features, the explainer, and the model family (Sections V-A and V-D).
4) **A scaling study.** On data generated from the AI4I 2020 process with up to $3 \cdot 10^5$ rows, we show how the power of each test changes with data size; the audit's runs are independent, so its cost grows roughly linearly and spreads across machines (Section V-E).
5) **A comparison with simpler audit designs.** Only the full audit rejects a model trained on shuffled labels, avoids optimistic verdicts, and still detects a strong shortcut at 300,000 rows (Section V-F).
6) **A reproducible benchmark** of seven model families with confidence intervals, and code that regenerates every number, table, and figure.

SHAP, oversampling, and shuffled-label nulls [7] are existing tools. What is new is their use per failure mechanism, the shortcut score, and the finding that the natural same-model baseline is fooled by how unusual mechanism rows are.

II. RELATED WORK AND BACKGROUND

This section reviews three lines of prior work: explaining models on AI4I 2020, evaluating explanations against a known answer, and randomization tests. Table II summarizes what each line of work does, what it cannot answer, and how our audit fills that gap.

A. Prior Work

Explanations on AI4I 2020. Matzka [6] introduced the dataset with local explanations of single predictions, and Zhao et al. [9] report global SHAP importance for tree ensembles on it. Neither checks the explanations against the mechanism flags or defines when an explanation counts as correct. For accuracy, Su and Tseng [16] report a positive-class F1 of 0.722 for a random forest; Khemani and Qureshi [17] report a LightGBM ROC-AUC of 0.973 under 5-fold cross-validation as part of a study on connected vehicles.

Evaluating explainers against a known answer. XAI-Bench [10], OpenXAI [11], BAM [12], Zhou et al. [18], and CLEVR-XAI [19] build data in which the important features are known, and ROAR [13] and ROAD [14] remove top-ranked features and measure the drop in accuracy. All of them ask whether an *explainer* reports what the model uses. We ask the reverse: given an exact explainer, does the *model* use what the physical process uses? In the terms of Chen et al. [20], we test whether explanations are "true to the data" for each failure type.

Randomization tests. Retraining on shuffled labels is a standard check of whether accuracy [21] or feature importance [7] beats chance, and Adebayo et al. [8] used model randomization to expose saliency maps that ignore the model (see also [22], [23]). We adopt the shuffled-label null of Altmann et al., but apply it to each mechanism's rows rather than to global importance.

Causal PdM. Dhande et al. [15] benchmark correlational models against a Bayesian structural causal model on data similar to AI4I 2020 and find that the causal model attributes HDF, PWF, and OSF failures correctly. We instead keep the kind of model most PdM work uses [24] and test its explanation.

B. SHAP and Its Limits

SHAP is based on the Shapley value from cooperative game theory, which splits a team's payout fairly among players by averaging each player's added value over all orders in which they could join. SHAP treats the features as players and the prediction as the payout [1]. For a model f and input x with d features, it assigns each feature j a value $\phi_j(x)$ such that $f(x) = \phi_0 + \sum_{j=1}^{d} \phi_j(x)$, where ϕ_0 is the average output. A positive value pushes the prediction toward failure, a negative

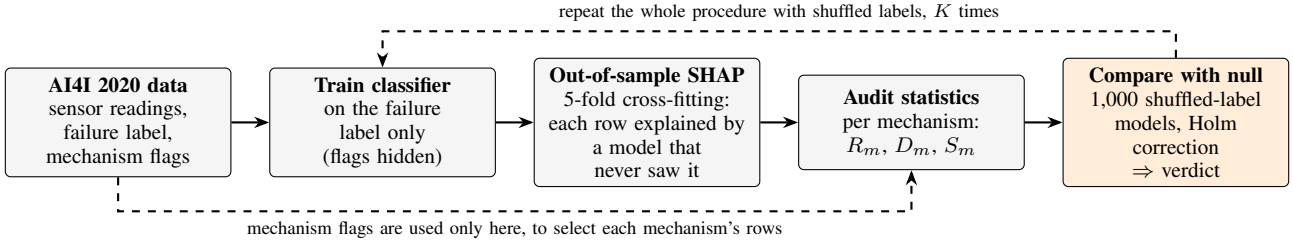


Fig. 1. Overview of the audit. The model never sees the mechanism flags. The flags only decide which rows each audit statistic looks at. The same pipeline, rerun on shuffled labels, tells us how large each statistic would be for a model that learned nothing about failure.

TABLE II
PRIOR LINES OF RESEARCH, WHAT THEY TEST, AND HOW OUR AUDIT DIFFERS.

Line of work	What is tested	Reference answer	Limitation for our question	What our audit adds
XAI on AI4I 2020 [6], [9]	the model	none	reports explanations but gives no pass/fail test	a pass/fail test per failure mechanism
Synthetic benchmarks [10]–[12]	the explainer	built into synthetic data	checks whether the explainer is accurate, not whether the model is right	tests the model, using an exact explainer for tree models
Feature removal [13], [14]	the explainer	none	measures accuracy loss, with no link to physical causes	compares credit with the physical trigger features
Randomization checks [7], [8]	the explainer or global importance	none	global, so it cannot say which feature explains which failure type	applies the shuffled-label idea to each mechanism's rows separately
Causal PdM [15]	causal vs. correlational models	assumed causal graph	attribution comes from a new causal model, not the model in use	keeps the usual model and tests its explanation

one away from it; we use the magnitude $|\phi_j(x)|$ as the size of the contribution.

TreeSHAP computes these values exactly for tree ensembles such as XGBoost and random forests [25]. It must evaluate the model as if some features were unknown. The default *path-dependent* version follows both branches of a split on an unknown feature, weighted by the training rows that went each way; the *interventional* version substitutes values from a background sample of real rows, which breaks correlations between features. The two can disagree when features are correlated [26], as our derived features are, and SHAP values depend on how missing features are modeled [27], [28] and can be manipulated [29]. We therefore repeat the audit with both versions (Section V-D).

III. METHODOLOGY

A. Problem Setting

Each row of the data has a sensor vector x_i, a binary failure label y_i, and a vector z_i of mechanism flags. The model sees only (x_i, y_i). For each mechanism m, domain knowledge gives a set E_m of *trigger features*, the sensors its physical rule depends on. The audit then asks a simple question: on the rows where mechanism m occurred, does the model's SHAP credit go to E_m, and more so than on other failures? We write $S_m = \{i : z_{i,m} = 1\}$ for the rows of mechanism m, $F = \{i : y_i = 1\}$ for all failure rows, and $\phi_j(x_i)$ for the SHAP value of feature j on row i.

B. Dataset and Failure Mechanisms

AI4I 2020 [6] contains 10,000 rows from a simulated milling process. Each row has a product quality type (`L`, `M`, or `H`), five process variables (air temperature, process temperature, rotational speed, torque, and tool wear), the failure label, and five mechanism flags. Only 339 rows (3.4%) are failures, so the classes are highly imbalanced.

The failure label is the logical OR of five mechanisms, listed in Table III. Four of them follow published rules on the sensor values. The fifth, random failure (RNF), happens with a small fixed probability and has no trigger features, so we treat it only descriptively.

The released labels do not follow the OR rule exactly: 24 rows carry several flags, 9 failures carry none, and 18 of the 19 RNF rows are not marked as failures. We therefore audit all flagged rows and, as a check, only the rows with exactly one flag.

C. Features

The trigger rules combine raw sensors, so we add four derived features computed from the raw values, not from the flags: *power* (torque × speed), *temperature difference* (process − air), *overstrain* (tool wear × torque), and *overstrain ratio* (overstrain divided by the type-specific limit). The mechanism flags are parts of the label, so they are never used as inputs.

Is the audit circular? AI4I 2020 is simulated, and its labels come from the same rules we audit against. We use it because real maintenance data rarely records the cause of each failure,

TABLE III
FAILURE MECHANISMS IN AI4I 2020 [6], THEIR TRIGGER RULES, AND THE TRIGGER FEATURES E_m USED BY THE AUDIT. COUNTS ARE AS RELEASED. THE TWF FAILURE POINT IS RANDOM WITHIN THE WINDOW.

Mech.	Trigger	E_m	n
TWF	tool wear at a random point in 200–240 min	tool wear	46
HDF	(process − air temp.) < 8.6 K and speed < 1380 rpm	both temps., their difference, speed	115
PWF	torque × speed (rad/s) outside [3500, 9000] W	torque, speed, power	95
OSF	tool wear × torque above 11,000/12,000/ 13,000 min·Nm for L/M/H	tool wear, torque, overstrain, its ratio	98
RNF	0.1% chance, independent of the process	–	19

TABLE IV
THE THREE AUDIT STATISTICS IN PLAIN TERMS. A MECHANISM PASSES THE AUDIT WHEN R_m AND D_m ARE SIGNIFICANTLY LARGER THAN UNDER THE NULL AND S_m IS NOT.

Statistic	Question it answers	Pass if
R_m, relevant share	On this mechanism's rows, how much of the credit goes to its trigger features?	large
D_m, specificity	Is that share larger on this mechanism's rows than on other failures?	large
S_m, shortcut score	Does some *other* feature gain credit specifically on this mechanism's rows?	not large

so an audit can only be validated where the causes are known. A simulator does not make the result automatic: a model can still fit the label through shortcuts or other features, and the audit rejects the shuffled-label model, flags an injected shortcut, and refuses to pass tool wear failure. Because the derived features could still make the audit easier, we also repeat it with raw features only (Section V-A).

D. Audit Statistics

For a set of rows S and a mechanism m, let $I_{m,j}(S) = |S|^{-1} \sum_{i \in S} |\phi_j(x_i)|$ be the average SHAP magnitude of feature j on those rows. The share of credit that goes to the trigger features is

$$r_m(S) = \sum_{j \in E_m} I_{m,j}(S) \Big/ \sum_{j=1}^{d} I_{m,j}(S). \tag{1}$$

From this we build three statistics:

$$R_m = r_m(S_m), \tag{2}$$

$$D_m = r_m(S_m) - r_m(F \setminus S_m), \tag{3}$$

$$S_m = \max_{j \notin E_m} \big[s_j(S_m) - s_j(F \setminus S_m) \big], \tag{4}$$

where $s_j(S) = I_{m,j}(S) / \sum_k I_{m,k}(S)$ is the share of a single feature j. Table IV explains what each statistic asks and when a mechanism passes.

Worked example. On the 95 power failure (PWF) rows, the average SHAP magnitudes of the three PWF trigger features are 4.96 (power), 1.93 (torque), and 1.33 (speed), which sum to 8.22. The twelve input features together sum to 11.82 (Fig. 3a). Hence $R_{\mathrm{PWF}} = 8.22/11.82 \approx 0.70$: about 70% of the model's credit on power failures goes to the features that physically cause them. Whether 70% is "high" is decided by the null in Section III-F, not by eye.

Each statistic covers a gap in the others. R_m alone can be high simply because the trigger features matter for all failures; D_m checks that the credit is specific to this mechanism. Both can still be high while a non-trigger feature also takes a large share; S_m catches this case, which is what a shortcut looks like. S_m is related to conditional randomization and knockoff tests [30], but it is computed from the explanation and only on the mechanism's rows.

E. Out-of-Sample Explanations

SHAP values computed on training rows describe what the model memorized about those rows, not what it learned in general. A model can fit its training failures closely and still explain new failures poorly. We therefore use cross-fitting [31]. We split the data into five folds of equal size, each with the same failure rate (*stratified*). For each fold, we train the full pipeline on the other four folds and compute SHAP values on the held-out fold. After five rounds, every row has a SHAP value from a model that never saw it during training, and the audit uses only these values. Section V-A shows that skipping this step changes the result.

F. Choosing the Right Null

Naive null. The obvious test compares R_m with the same statistic on random failure rows of equal size under the same model. This test is flawed. Rows of a mechanism are selected by extreme values of its trigger features, and SHAP values tend to be large at extreme values even for a model that learned nothing. We confirm this in Section V-B.

Shuffled-label null. We instead compare the real model with models that cannot know anything about failure. To build one, we randomly reorder the failure labels across rows. This keeps the sensor values, the failure rate, and the whole training pipeline unchanged, but it destroys any real link between the sensors and failure. The mechanism flags stay attached to their rows, so the audit still looks at the same rows. We then rerun the full procedure (features, oversampling, training, and cross-fitting) on the shuffled labels, keeping the hyperparameters tuned on the real labels, and compute all three statistics. Repeating this K times gives K values of each statistic that show how large it can be by chance, including the effect of unusual feature values that breaks the naive null.

For a statistic T of the real model with null values $T^{(1)}, \ldots, T^{(K)}$, the p-value is

$$p = \frac{1 + \#\{k : T^{(k)} \geq T\}}{K + 1}, \tag{5}$$

the fraction of null models that score at least as high as the real model, counting the real model once so that p is never zero.

With $K = 1{,}000$, the smallest possible value is $1/1{,}001 \approx 0.001$, reached when no null model scores as high as the real one.

Because we test four mechanisms, one could pass by chance. Holm's method [32] guards against this: it sorts the four p-values, multiplies the smallest by 4, the next by 3, and so on, which keeps the chance of any false pass below 5%. A mechanism passes a test if its adjusted p is below 0.05; for example, an unadjusted 0.001 becomes 0.004 (Table VII).

Random failures. RNF has no trigger features. We only report whether its credit is more concentrated on a single feature than under the null.

G. Model Training and Selection

We split the data 70/20/10 into training, validation, and test sets, stratified by label, and fit scaling and one-hot encoding on the training set only, so that no information from the validation or test rows leaks into training.

With only 3.4% failures, a model can reach 96.6% accuracy by never predicting failure, so we *oversample* the failure class in the training data. SMOTE [33] creates a synthetic failure row by picking a real failure row and moving it part of the way toward one of its nearest failure neighbors. Borderline-SMOTE [34] does this only for failure rows that lie close to non-failure rows, where the model is most likely to make mistakes. ADASYN [35] creates more synthetic rows where failures are hardest to learn. We compare the three using the validation F1 of a default XGBoost [36] model and keep the best (Section IV-A).

We train seven model families: logistic regression, a support vector machine (SVM) with a radial basis function kernel, k-nearest neighbors (k-NN, $k = 7$), a decision tree (depth 10), a random forest (300 trees, depth 12), XGBoost (300 trees, depth 6, learning rate 0.1), and a multilayer perceptron (MLP) with hidden layers of 64 and 32 units. Other hyperparameters are library defaults (scikit-learn 1.9.1, XGBoost 3.4.1, PyTorch 2.14.0), and the random seed is 42.

XGBoost has the highest validation F1, so we tune it with a 40-iteration randomized search under 5-fold stratified cross-validation, scored on F1. Borderline-SMOTE runs inside the search pipeline, so each fold oversamples only its own training part. The search covers depth 3–8, 200–600 trees, learning rate 0.01–0.2, row and column subsampling 0.6–1.0, minimum child weight 1–5, and γ 0–0.3. The model outputs a failure probability for each row, and a row raises an alarm when this probability exceeds a *decision threshold*. We choose the threshold that maximizes F1 on the validation set.

Evaluation metrics. Because failures are rare, we report metrics for the failure class. *Precision* is the share of alarms that are real failures, *recall* the share of real failures that raise an alarm, and *F1* their harmonic mean. *ROC-AUC* measures how well the model ranks failures above non-failures over all thresholds. The *Brier score* (mean squared error of the predicted probability) and the *expected calibration error* (ECE, the average gap between predicted and observed failure rates) check whether probabilities can be taken at face value; lower is better for both.

TABLE V
EXPERIMENTAL PLAN.

Experiment	Question	K
Primary audit, seed 42	Do the explanations match each mechanism? (H1)	1,000
Seeds 1–4	Is the result stable across seeds?	200
Single-flag rows	Do rows with several flags change the result?	1,000
Raw features only	Do the derived features make the audit circular?	1,000
In-sample SHAP	Does explaining training rows inflate the result?	1,000
Interventional TreeSHAP	Does the result depend on the explainer?	200
Random forest	Does it depend on the model family?	200
Shuffled-label model	Sanity check; does the naive null fail? (H3)	200
Shortcut sweep	When is an injected shortcut caught? (H2)	500
Scaling, 10^4–$3 \cdot 10^5$ rows	How do power and cost change with size?	200

A small test set makes every metric uncertain, so we report 95% confidence intervals (CIs): exact binomial (Clopper–Pearson) intervals for precision and recall, and intervals from 5,000 stratified bootstrap resamples of the test set otherwise. We also run 5-fold cross-validation of the whole pipeline at a threshold of 0.5. The hyperparameters are tuned on the training split that this cross-validation reuses, so its F1 is slightly optimistic.

H. Experimental Plan

Table V lists every experiment, the question it answers, and the number K of shuffled-label models in its null. The primary audit uses $K = 1{,}000$, so the smallest possible p-value before correction is 0.001; it takes 12 minutes on eight CPU cores.

Explainer and model variants. The interventional runs use backgrounds of 100, 500, and 1,000 training rows per fold; their null runs use the same explainer, and the random-forest null uses the random forest.

Shortcut sensor. To test H2, we add a fake sensor $s = z_{\mathrm{HDF}} + \mathcal{N}(0, \sigma^2)$, a noisy copy of the heat dissipation flag with no physical meaning, and vary the noise σ from 2 down to 0.1. A smaller σ makes the sensor a stronger shortcut. The sensor is not in E_{HDF}, so a model that relies on it should fail the HDF audit.

Shuffled-label model. A model trained on shuffled labels is interchangeable with the null runs, so we use it only as a check of the code and as the case the naive null should reject.

Generated data for scaling. To study larger data, we generate new rows from the process documented with AI4I 2020 [6], copying no released row. The sensor distributions follow the documentation and the release; tool wear grows until the tool reaches a life drawn from $U(200, 240)$ min, where it fails (TWF) with probability 0.4; and HDF, PWF, and OSF follow the published rules, which reproduce the released flags exactly.

TABLE VI
FAILURE-CLASS TEST METRICS, SORTED BY F1, WITH BOOTSTRAP 95% CI FOR F1. ROC-AUC: AREA UNDER THE RECEIVER OPERATING CHARACTERISTIC CURVE.

Model	Prec.	Rec.	F1 (95% CI)	ROC-AUC
XGBoost (tuned)	**1.00**	0.79	**0.89** (0.79–0.95)	0.988
XGBoost	0.86	0.88	0.87 (0.78–0.94)	0.988
Random forest	0.79	0.88	0.83 (0.74–0.92)	**0.989**
MLP	0.65	**0.91**	0.76 (0.67–0.84)	0.988
Decision tree	0.68	0.82	0.75 (0.64–0.84)	0.927
SVM	0.42	0.88	0.57 (0.48–0.65)	0.976
k-NN	0.43	0.79	0.56 (0.46–0.65)	0.877
Logistic regression	0.24	0.85	0.37 (0.31–0.43)	0.943

The generated failure rate is 3.5–4.0%, against 3.4% in the release. We audit $n \in \{10^4, 3 \cdot 10^4, 10^5\}$ with three generation seeds and $3 \cdot 10^5$ with one, each without the shortcut sensor and with it at $\sigma \in \{0.35, 0.25\}$. The full generator is in the released code.

IV. RESULTS

A. Predictive Performance

Oversampler. On the validation set, Borderline-SMOTE gives the highest F1 (0.826, against 0.768 for SMOTE and 0.747 for ADASYN). The other two reach slightly higher recall but lose more precision, so we use Borderline-SMOTE in all later experiments.

Model comparison. Table VI reports test results for all models. Tree ensembles clearly outperform the linear and distance-based models. The tuned XGBoost reaches F1 0.89 at its validation-selected threshold of 0.978, against 0.87 for the default XGBoost at 0.5. The confidence intervals overlap, so we cannot claim that tuning or the threshold helps.

Uncertainty. The test set holds only 34 failures, so the intervals are wide. The tuned model raises 27 alarms, all of them real failures, and misses 7 failures. This gives precision 1.00 (95% CI 0.87–1.00) and recall 0.79 (0.62–0.91). Its probabilities are well calibrated: the Brier score is 0.0090, against 0.033 for always predicting the base rate, and the expected calibration error is 0.008.

Comparison with prior work. Five-fold cross-validation of the full pipeline gives a failure-class F1 of 0.822 ± 0.013 and a macro-F1 of 0.908 ± 0.007. Both this and our single-split F1 of 0.89 are above the 0.722 reported by Su and Tseng [16] on the same binary task. The model is therefore accurate enough to be a realistic target for the audit.

B. Main Audit of the Tuned Model (H1)

Fig. 2 is the central result of the paper. Each gray dot is one of the 1,000 models trained on shuffled labels; together they show how large R_m and D_m can be by chance. The orange dot is the real model. A mechanism passes when the orange dot sits clearly above the gray cloud.

Heat dissipation, power, and overstrain failure pass. For these three mechanisms the real model lies above every shuffled-label model on both statistics, and all adjusted p-values are 0.004 (Table VII). On their rows, the share of credit

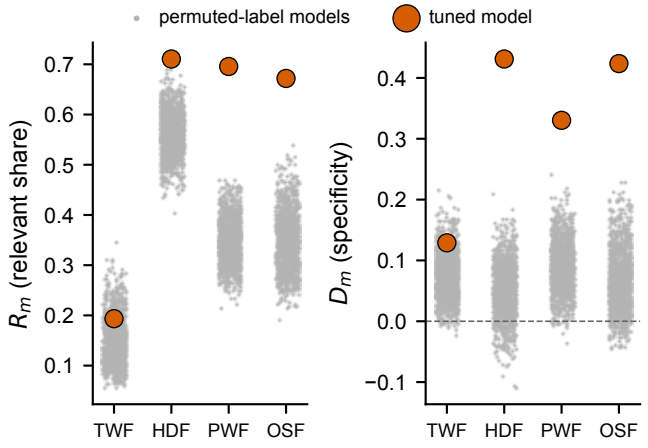


Fig. 2. Main audit. Out-of-sample relevant share R_m (left) and specificity D_m (right) of the tuned model (orange) against 1,000 shuffled-label models (gray). For HDF, PWF, and OSF the real model lies above every null model on both statistics. For TWF it lies inside the null cloud.

on the trigger features exceeds the share on other failures by 0.33 to 0.43, while the null averages stay below 0.09. The shortcut score flags nothing, and the top feature of each mechanism is one of its trigger features.

Tool wear failure does not pass. TWF fails both tests ($p = 0.201$ and 0.077). Tool wear is still its top feature, but it takes only 19% of the credit on TWF rows, against 15% under the null. This is not surprising: the TWF rule fires at a *random* point inside a tool-wear window, so tool wear only partly determines it, and 46 rows give the model little to learn from.

Fig. 3 shows why a per-mechanism view is needed. The heatmap (a) shows the average SHAP magnitude of each feature on each mechanism's rows. Each physical mechanism lights up its own trigger features: speed and temperature difference for HDF, power and torque for PWF, and the overstrain ratio for OSF. The usual global summary (b) ranks speed, power, and tool wear highest overall, but it mixes all failures together and cannot tell which feature explains which failure.

V. EXPERIMENTAL ANALYSIS

We now change the choices behind the main audit one at a time, test the naive baseline, inject a shortcut, and scale up the data. Table VIII collects the p-values of all variants.

A. Robustness of the Main Audit (H1)

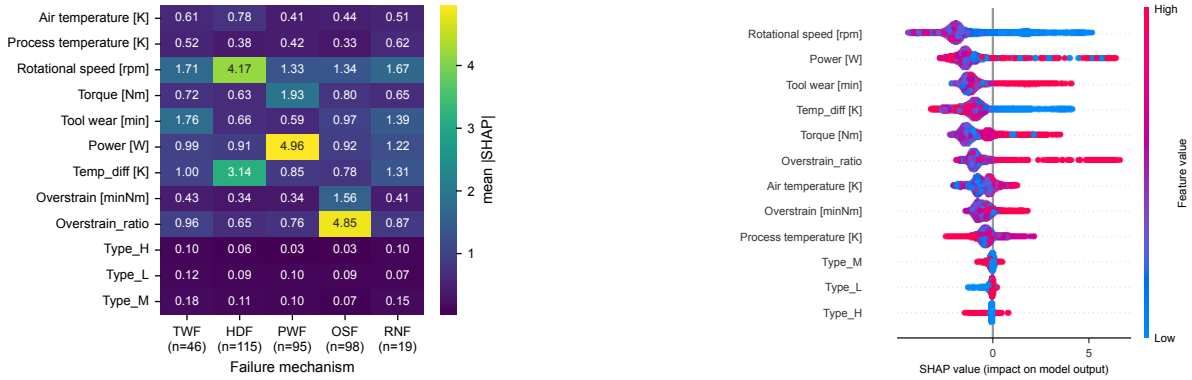
Seeds and rows. The HDF, PWF, and OSF verdicts hold in all five seeds and on rows with a single flag. TWF fails both tests in every seed.

In-sample explanations. When SHAP values come from a model that has seen every audited row, all four mechanisms pass, including TWF ($p = 0.004$; $R_{\mathrm{TWF}} = 0.35$ against 0.19 out of sample). In-sample explanations can therefore make a model look better aligned than it is, which is why the audit uses cross-fitting.

TABLE VII

MAIN AUDIT (CROSS-FITTED, SEED 42, $K = 1{,}000$). NULL: MEAN OVER SHUFFLED-LABEL MODELS. p: HOLM-ADJUSTED OVER THE FOUR PHYSICAL MECHANISMS; $^{\times}$ MARKS A FAILED R_m OR D_m TEST ($p \geq 0.05$); S_m (EQ. (4)) FLAGS A SHORTCUT IF $p < 0.05$. FOR RNF THE STATISTIC IS THE LARGEST FEATURE'S SHARE, AND p TESTS FOR MORE CONCENTRATION THAN THE NULL.

	n	Top feature	R_m	Null	p	D_m	Null	p	S_m	Null	p
TWF	46	tool wear	0.193	0.152	0.201$^{\times}$	0.129	0.070	0.077$^{\times}$	0.025	0.025	1.000
HDF	115	speed	0.711	0.557	0.004	0.431	0.049	0.004	0.001	0.015	1.000
PWF	95	power	0.696	0.347	0.004	0.330	0.088	0.004	0.001	0.011	1.000
OSF	98	overstrain ratio	0.672	0.345	0.004	0.424	0.067	0.004	-0.001	0.019	1.000
RNF	19	speed	0.186	0.240	0.985	–	–	–	–	–	–



(a) Mean out-of-sample $|\phi_j|$ on the rows of each mechanism

(b) Global SHAP summary on all rows

Fig. 3. SHAP explanations of the tuned model. (a) Per-mechanism view used by the audit: each mechanism's credit concentrates on its own trigger features (bright cells). (b) The standard global view ranks features overall but does not separate the mechanisms.

TABLE VIII

HOLM-ADJUSTED AUDIT p-VALUES FOR R_m, D_m, AND S_m ACROSS ANALYSIS VARIANTS. $^{\times}$ MARKS A FAILED R OR D TEST ($p \geq 0.05$) AND † A FLAGGED SHORTCUT ($p_S < 0.05$); NO VARIANT FLAGS ONE. INTERVENTIONAL ROWS USE BACKGROUND SEED 0; SEEDS 1 AND 2 GIVE THE SAME VERDICTS. THE SMALLEST ATTAINABLE ADJUSTED p IS $4/(K+1)$. THE SHUFFLED-LABEL MODEL IS INTERCHANGEABLE WITH ITS NULL AND SERVES ONLY AS A SANITY CHECK; THE LAST TWO ROWS APPLY THE NAIVE NULL OF SECTION III-F.

		TWF			HDF			PWF			OSF		
Variant	K	R	D	S	R	D	S	R	D	S	R	D	S
Primary, seed 42	1000	0.201$^{\times}$	0.077$^{\times}$	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
single-mechanism rows	1000	0.187$^{\times}$	0.076$^{\times}$	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
Seed 1	200	0.199$^{\times}$	0.119$^{\times}$	0.776	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 2	200	0.149$^{\times}$	0.065$^{\times}$	0.498	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 3	200	0.174$^{\times}$	0.095$^{\times}$	0.736	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 4	200	0.269$^{\times}$	0.129$^{\times}$	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Raw features only	1000	0.345$^{\times}$	0.616$^{\times}$	1.000	0.190$^{\times}$	0.004	1.000	0.004	0.004	1.000	0.004	0.048	1.000
In-sample SHAP	1000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
Interventional, background 100	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Interventional, background 500	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Interventional, background 1000	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Random forest	200	0.617$^{\times}$	0.398$^{\times}$	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.040	0.020	1.000
Permuted labels (sanity)	200	1.000$^{\times}$	0.731$^{\times}$	1.000	0.199$^{\times}$	0.557$^{\times}$	1.000	1.000$^{\times}$	0.731$^{\times}$	1.000	1.000$^{\times}$	0.731$^{\times}$	0.637
Naive null, tuned model	–	0.002	–	–	0.002	–	–	0.002	–	–	0.002	–	–
Naive null, permuted labels	–	0.002	–	–	0.002	–	–	0.002	–	–	0.002	–	–

Raw features only. Without the derived features, PWF and OSF still pass both tests (OSF specificity narrowly, $p = 0.048$). HDF passes only on specificity, and TWF on neither. The derived features help the HDF verdict but are not needed for PWF and OSF, so the audit is not circular for those two. HDF is defined by a *difference* of two temperatures, which a tree model can only approximate with many splits, so it is harder to audit from raw inputs.

B. Is the Naive Null Enough? (H3)

Under the shuffled-label null, the shuffled-label model fails every test, as it should. Under the naive null, however, the same model passes on all four mechanisms (adjusted $p = 0.002$), exactly as the real model does, and even TWF passes, which

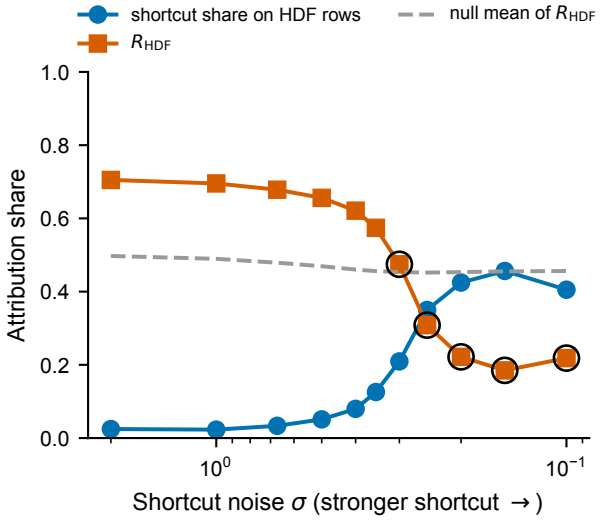


Fig. 4. Shortcut sweep. As the noise σ shrinks (left to right), the fake sensor takes more of the credit on HDF rows (blue) and the share on the true trigger features falls (orange). Circles mark noise levels at which the HDF relevant-share test fails.

the shuffled-label null rejects. The naive null therefore cannot tell a trained model from one that learned nothing, which supports H3: a retrained null is necessary. Any subgroup-level test of explanations needs a null that accounts for how unusual the audited rows are.

C. Detecting an Injected Shortcut (H2)

Fig. 4 shows what happens as the fake sensor becomes a stronger shortcut (left to right): credit moves from the real trigger features (orange) to the fake sensor (blue).

With $K = 500$ null models, the audit's response falls into three stages:

- **Weak shortcut ($\sigma \geq 0.4$).** The sensor takes at most 8% of the credit and the audit passes. This is correct, because the model barely uses it.
- **Moderate shortcut ($\sigma = 0.35$ to 0.3).** At 13% the relevant-share test still passes narrowly ($p = 0.044$). At 21% ($\sigma = 0.3$) it fails.
- **Strong shortcut ($\sigma \leq 0.25$).** At 35% the shortcut score also flags the sensor, and from 42% ($\sigma = 0.2$) the specificity test fails as well.

Throughout the sweep, test F1 stays between 0.84 and 0.92. Accuracy alone would never reveal that the model has started to rely on a meaningless sensor. On the released data, H2 holds once a shortcut takes about a fifth of the credit, but not for lighter use. PWF and OSF pass at every noise level, so the audit does not raise false alarms on unaffected mechanisms. TWF stays near the decision boundary: its specificity test passes narrowly at four levels ($0.030 \leq p \leq 0.0499$) and fails at the other seven.

The shortcut score is conservative here because the null models see the sensor too and give it some credit on HDF rows, where its values are unusual (null mean of S_{HDF}: 0.09).

D. Explainer and Model Variants

The HDF, PWF, and OSF verdicts hold with interventional TreeSHAP and with a random forest (Table VIII). TWF does not behave consistently. With interventional TreeSHAP it passes both tests at every background size and seed (R_{TWF} between 0.45 and 0.47, tool wear its top feature). With the random forest it fails both ($R_{\mathrm{TWF}} = 0.13$, speed its top feature).

A likely reason is correlation. Tool wear has a Spearman correlation of 0.92 with both overstrain features, so they almost always rise and fall together. Path-dependent TreeSHAP spreads credit over features correlated with the ones a tree splits on, while the interventional version breaks that dependence [26]. An audit verdict is therefore a statement about a model *and* an explainer together. A mechanism that passes under only one explainer should be reported as unresolved.

E. Scaling to Larger Data (H1, H2)

Table X repeats the audit on generated data of increasing size. Without the shortcut sensor, HDF, PWF, and OSF pass both tests and are never flagged at any size or seed. TWF fails the relevant-share test in every run, so more data alone does not resolve it. The shortcut score flags TWF in 3 of 10 runs without the sensor, each time for process temperature, which plays no role in tool wear failure; the effect is small ($S_{\mathrm{TWF}} \leq 0.05$).

The two shortcut tests behave differently as data grows. At $\sigma = 0.25$ the sensor takes 27–45% of the HDF credit. The relevant-share test flags it in all nine runs up to 100,000 rows, but not at 300,000 rows. There, $R_{\mathrm{HDF}} = 0.472$ still exceeds the 95th null percentile of 0.417: with more data the null distribution becomes narrow, and "better than a model that learned nothing" becomes an easy bar to clear. The shortcut score flags the sensor in all ten runs, including at 300,000 rows ($S_{\mathrm{HDF}} = 0.249$ against a 95th percentile of 0.211). Neither test reliably flags the lighter shortcut at $\sigma = 0.35$ (6–20% of the credit); only the relevant-share test does so, in two runs at 10,000 rows. H2 therefore holds at every size for a shortcut that takes about a third of the credit, provided the audit includes the shortcut score.

Cost. The audit costs $5(K+1)$ model fits. One cross-fitted run takes 1.3 s at 10^4 rows, 31 s at $3 \cdot 10^5$, and 115 s at 10^6 rows on one core, roughly linear in n. On eight cores, an audit with $K = 200$ took 3 minutes at 10^4 rows and about an hour at $3 \cdot 10^5$. The runs are independent, so the work can be spread across machines.

F. Comparison with Simpler Audit Designs

No published method audits explanations against failure mechanisms, and the benchmarks in Table II test explainers on data built for them. We therefore compare the full audit with four simpler designs a practitioner might use, each missing one ingredient.

We judge each design by three basic checks: (1) it should reject a model trained on shuffled labels, which knows nothing about failure; (2) it should not be optimistic about TWF, whose

TABLE IX

COMPARISON OF AUDIT DESIGNS ON THREE BASIC CHECKS. EACH SIMPLER DESIGN REMOVES ONE INGREDIENT OF THE FULL AUDIT. VALUES ARE HOLM-ADJUSTED p-VALUES (HDF FOR CHECK 3) OR RUN COUNTS FROM TABLES VIII AND X AND ONE ADDITIONAL EXPERIMENT PER REMAINING CELL.

Design	1. Rejects shuffled-label model?	2. Avoids optimism on TWF?	3. Catches shortcut at $3 \cdot 10^5$ rows?
Global SHAP ranking (common practice [9])	no verdict	no verdict	no verdict
Naive null (random failure rows, same model)	**No**: all pass ($p = 0.002$)	**No**: passes ($p = 0.002$)	**No**: HDF passes ($p = 0.002$)
In-sample SHAP with shuffled-label null	Yes: all fail ($p \geq 0.22$)	**No**: passes ($p = 0.004$)	**No**: R, D pass; $p_S = 0.06$
Relevant share R_m only (out-of-sample, shuffled null)	Yes	Yes ($p = 0.201$)	**No**: 0 of 1 runs
Full audit (R_m, D_m, S_m, out-of-sample, shuffled null)	**Yes**: all fail	**Yes**: fails ($p = 0.201$, 0.077)	**Yes**: 1 of 1 runs

TABLE X

AUDIT ON DATA GENERATED FROM THE AI4I 2020 PROCESS AT SIZE n ($K = 200$): MEAN SENSOR SHARE OF THE HDF CREDIT, AND THE FRACTION OF GENERATION SEEDS IN WHICH THE HDF SHORTCUT IS FLAGGED BY THE RELEVANT-SHARE TEST ($p_R \geq 0.05$) OR THE SHORTCUT SCORE ($p_S < 0.05$), TWF PASSES THE SPECIFICITY TEST, OR THE SHORTCUT SCORE FLAGS TWF. MIN.: WALL-CLOCK MINUTES FOR ONE AUDIT ON EIGHT CORES. HDF, PWF, AND OSF PASS THE R AND D TESTS IN EVERY RUN WITHOUT THE SENSOR.

				HDF flagged by		TWF		
n	σ	Seeds	Sensor	R	S	D pass	S flag	Min.
10,000	–	3	0.000	0/3	0/3	0/3	0/3	3
	0.35	3	0.158	2/3	0/3	0/3	0/3	3
	0.25	3	0.390	3/3	3/3	0/3	0/3	3
30,000	–	3	0.000	0/3	0/3	0/3	1/3	7
	0.35	3	0.128	0/3	0/3	2/3	0/3	6
	0.25	3	0.346	3/3	3/3	2/3	1/3	7
100,000	–	3	0.000	0/3	0/3	1/3	2/3	17
	0.35	3	0.100	0/3	0/3	2/3	1/3	19
	0.25	3	0.315	3/3	3/3	2/3	1/3	19
300,000	–	1	0.000	0/1	0/1	0/1	0/1	52
	0.35	1	0.063	0/1	0/1	0/1	0/1	59
	0.25	1	0.271	0/1	1/1	0/1	1/1	61

held-out credit on tool wear (19%) is close to chance (15%); and (3) it should flag a shortcut that takes 27% of the HDF credit at 300,000 rows.

Table IX shows that each simpler design fails at least one check, for a clear reason. The global ranking gives no verdict and mixes all failure types (Fig. 3b). The naive null keeps the same model, so it cannot separate learning from the unusual values of mechanism rows; it approves both the shuffled-label model and the shortcut model. In-sample SHAP lets the model explain memorized rows: it raises R_{TWF} from 0.19 to 0.35, and at 300,000 rows it misses the shortcut ($p_S = 0.06$), which the out-of-sample shortcut score flags. R_m alone only checks the expected features; with more data its null narrows and a 27% shortcut clears the bar. Each ingredient of the full audit removes one of these causes.

G. Limitations

Simulated data. AI4I 2020 is simulated from the same rules we use as ground truth. The audit therefore checks agreement with the simulator's logic, not with measurements from real machines; we contribute a protocol, not a finding about real equipment. Autran et al. [37] show that realistic irregularities added to AI4I 2020 change model performance, so results on the clean data may not transfer.

Scope of the audit. The audit misses a shortcut that takes only a tenth of the credit, and we test only one kind of shortcut, a noisy copy of one mechanism's flag. It needs the trigger features E_m to be known. We study two model families and two explainers on one simulator, and mechanisms with few rows, such as TWF and RNF, give weak evidence either way.

VI. CONCLUSION

We asked whether the SHAP explanations of a predictive maintenance classifier point to the true physical causes of failure. Using the AI4I 2020 dataset, whose failure mechanisms are recorded but hidden from the model, we built an audit that combines out-of-sample SHAP values, three simple statistics, and a null made of models trained on shuffled labels. For heat dissipation, power, and overstrain failure, the tuned model's explanations match the physical mechanism in all five seeds, with two explainers and two model families. For tool wear failure, the verdict depends on the explainer and remains unresolved. For trustworthy AI in safety-critical cyber-physical systems, this gives a concrete test that an explanation can pass or fail. Unlike simpler designs, the full audit rejects a meaningless model, avoids optimistic verdicts, and still catches a strong shortcut on large data.

Three lessons go beyond this dataset: high accuracy does not guarantee correct explanations; a same-model baseline approves a model that learned nothing; and an audit should look for unexpected features, not only expected ones.

The audit applies wherever maintenance records log a root cause for each failure. Next steps are real maintenance logs with recorded root causes, process data with labeled fault types such as the extended Tennessee Eastman simulation [38], [39], shortcuts other than noisy label copies, and a formal power analysis of the detection threshold.

ACKNOWLEDGMENT

Portions of the manuscript text and the analysis code were drafted with the assistance of an AI system (Claude, Anthropic). The authors reviewed and verified all content and take full responsibility for it. Code, model artifacts, and scripts

that regenerate every number, table, and figure are available at https://github.com/kowshik24/glowing-broccoli.

REFERENCES

[1] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] S. Vollert, M. Atzmueller, and A. Theissler, "Interpretable machine learning: A brief survey from the predictive maintenance perspective," in *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2021, pp. 1–8.

[3] L. Cummins, A. Sommers, S. Bakhtiari Ramezani, S. Mittal, J. Jabour, M. Seale, and S. Rahimi, "Explainable predictive maintenance: A survey of current methods, challenges and opportunities," *IEEE Access*, vol. 12, pp. 57 574–57 602, 2024.

[4] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[5] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking Clever Hans predictors and assessing what machines really learn," *Nature Communications*, vol. 10, no. 1, p. 1096, 2019.

[6] S. Matzka, "Explainable artificial intelligence for predictive maintenance applications," in *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*. IEEE, 2020, pp. 69–74.

[7] A. Altmann, L. Tološi, O. Sander, and T. Lengauer, "Permutation importance: A corrected feature importance measure," *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, 2010.

[8] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[9] W. Zhao, J. Ding, X. Huang, and Y. Zhang, "Research on milling machine predictive maintenance based on machine learning and SHAP analysis in intelligent manufacturing environment," arXiv:2512.01205, 2025, accepted at ICEIEC 2025.

[10] Y. Liu, S. Khandagale, C. White, and W. Neiswanger, "Synthetic benchmarks for scientific research in explainable machine learning," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[11] C. Agarwal, S. Krishna, E. Saxena, M. Pawelczyk, N. Johnson, I. Puri, M. Zitnik, and H. Lakkaraju, "OpenXAI: Towards a transparent evaluation of model explanations," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 35, 2022.

[12] M. Yang and B. Kim, "Benchmarking attribution methods with relative feature importance," *arXiv preprint arXiv:1907.09701*, 2019.

[13] S. Hooker, D. Erhan, P.-J. Kindermans, and B. Kim, "A benchmark for interpretability methods in deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 9737–9748.

[14] Y. Rong, T. Leemann, V. Borisov, G. Kasneci, and E. Kasneci, "A consistent and efficient evaluation strategy for attribution methods," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 18 770–18 795.

[15] S. Dhande, C. Ma, G. P. Saggese, P. Smith, and K. Taduri, "A benchmark of causal vs. correlation AI for predictive maintenance," arXiv:2512.01149, 2026.

[16] Y.-K. Su and C.-J. Tseng, "A physics-informed benchmarking framework for machine learning and tree-based ensembles in IIoT-enabled predictive maintenance," *Sensors*, vol. 26, no. 16, p. 5026, 2026.

[17] K. Khemani and A. N. Qureshi, "AI-driven predictive maintenance for connected vehicles using environmental context integration evaluated through simulation benchmarking and field validation," *Discover Vehicles*, vol. 2, p. 19, 2026.

[18] Y. Zhou, S. Booth, M. T. Ribeiro, and J. Shah, "Do feature attribution methods correctly attribute features?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 9, 2022, pp. 9623–9633.

[19] L. Arras, A. Osman, and W. Samek, "CLEVR-XAI: A benchmark dataset for the ground truth evaluation of neural network explanations," *Information Fusion*, vol. 81, pp. 14–40, 2022.

[20] H. Chen, J. D. Janizek, S. Lundberg, and S.-I. Lee, "True to the model or true to the data?" *arXiv preprint arXiv:2006.16234*, 2020.

[21] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *Journal of Machine Learning Research*, vol. 11, no. 62, pp. 1833–1863, 2010. [Online]. Available: https://www.jmlr.org/papers/v11/ojala10a.html

[22] P.-J. Kindermans, S. Hooker, J. Adebayo, M. Alber, K. T. Schütt, S. Dähne, D. Erhan, and B. Kim, "The (un)reliability of saliency methods," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*, ser. Lecture Notes in Computer Science, W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Müller, Eds. Springer, 2019, vol. 11700, pp. 267–280.

[23] G. Yona and D. Greenfeld, "Revisiting sanity checks for saliency maps," in *NeurIPS 2021 Workshop on eXplainable AI Approaches for Debugging and Diagnosis (XAI4Debugging)*, 2021. [Online]. Available: https://arxiv.org/abs/2110.14297

[24] O. Serradilla, E. Zugasti, J. Rodriguez, and U. Zurutuza, "Deep learning models for predictive maintenance: A survey, comparison, challenges and prospects," *Applied Intelligence*, vol. 52, no. 10, pp. 10 934–10 964, 2022.

[25] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[26] K. Aas, M. Jullum, and A. Løland, "Explaining individual predictions when features are dependent: More accurate approximations to Shapley values," *Artificial Intelligence*, vol. 298, p. 103502, 2021.

[27] D. Janzing, L. Minorics, and P. Blöbaum, "Feature relevance quantification in explainable AI: A causal problem," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, S. Chiappa and R. Calandra, Eds., vol. 108. PMLR, 2020, pp. 2907–2916.

[28] I. E. Kumar, S. Venkatasubramanian, C. Scheidegger, and S. Friedler, "Problems with Shapley-value-based explanations as feature importance measures," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 5491–5500.

[29] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. ACM, 2020, pp. 180–186.

[30] E. Candès, Y. Fan, L. Janson, and J. Lv, "Panning for gold: 'model-X' knockoffs for high dimensional controlled variable selection," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 80, no. 3, pp. 551–577, 2018.

[31] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins, "Double/debiased machine learning for treatment and structural parameters," *The Econometrics Journal*, vol. 21, no. 1, pp. C1–C68, 2018.

[32] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[33] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[34] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *International Conference on Intelligent Computing*. Springer, 2005, pp. 878–887.

[35] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE International Joint Conference on Neural Networks*. IEEE, 2008, pp. 1322–1328.

[36] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[37] J.-V. Autran, V. Kuhn, J.-P. Diguet, M. Dubois, and C. Buche, "AI4I-PMDI: Predictive maintenance datasets with complex industrial settings' irregularities," *Procedia Computer Science*, vol. 246, pp. 1201–1209, 2024.

[38] C. A. Rieth, B. D. Amsel, R. Tran, and M. B. Cook, "Additional Tennessee Eastman process simulation data for anomaly detection evaluation," Harvard Dataverse, V1, 2017.

[39] ——, "Issues and advances in anomaly detection evaluation for joint human-automated systems," in *Advances in Human Factors in Robots and Unmanned Systems*, ser. Advances in Intelligent Systems and Computing, J. Chen, Ed., vol. 595. Springer, 2018, pp. 52–63.